UNIVERSIDAD DE SANTIAGO DE CHILE

FACULTAD DE INGENIERÍA

Departamento de Ingeniería Mecánica

Informe de proyecto: Manipulador robótico

Fundamentos de Robótica

Integrantes: Alejandro Pavez, Luciana Bravo y Diego Zúñiga.
Carrera: Ingeniería Civil Mecatrónica.
Docente: Diego Luna Segura
Sección: L-1
Fecha de entrega: 06 de Noviembre del 2026

Santiago – Chile
2026

Índice

1. Resumen **1**

2. Introducción **1**

3. Marco teórico **2**

3.1. Manipuladores robóticos y grados de libertad 2

3.2. Cinemática directa e inversa 2

3.3. Modelo Denavit–Hartenberg 3

4. Presentación del problema **3**

4.1. Requerimientos del sistema 4

5. Solución del problema **4**

5.1. Elección y construcción del manipulador 4

5.2. Elección de materiales y componentes 5

5.3. Integración electrónica 8

5.4. Arquitectura del sistema de control 10

5.5. Interfaz de visualización y rutinas automáticas 11

5.6. Organización del equipo 14

6. Conclusiones **14**

Referencias **16**

Anexo I: Rutina de código **17**

Anexo II: Desarrollo del algoritmo Denavit–Hartenberg **19**

Anexo III: Repositorio GitHub **22**

Índice de figuras

1. Vista general del manipulador robótico construido. 2
2. Vista general del manipulador robótico construido. 5
3. Servomotores utilizados en el manipulador robótico. 6
4. Microcontrolador ESP32 y módulo controlador PWM PCA9685. 6
5. Sensor ultrasónico HC-SR04 utilizado para la detección de objetos. 7
6. Fuente de alimentación externa de 5 V y 10 A utilizada en el sistema. 8
7. Montaje electrónico del manipulador robótico, con distribución de conexiones y alimentación. 9
8. Detalle del módulo PCA9685 y de las conexiones con la fuente de alimentación. 9
9. Diagrama UML de clases de la aplicación desarrollada para el manipulador robótico. . . 11
10. Interfaz de visualización en Python, con modelo tridimensional, parámetros D–H y matrices de transformación. 11
11. Interfaz móvil Blynk IoT utilizada para interactuar con el manipulador. 12
12. Panel web de Blynk IoT con controles articulares y selección de rutinas. 12
13. Manipulador robótico durante la rutina de manipulación y apilamiento de bloques. . . . 13

Índice de tablas

1. Parámetros del método Denavit–Hartenberg. 3
2. Distribución de responsabilidades del equipo de trabajo. 14
3. Dimensiones utilizadas en el modelo cinemático. 19
4. Parámetros D–H utilizados en el manipulador robótico. 19

1. Resumen

El presente proyecto consistió en el desarrollo e implementación de un manipulador robótico a escala, capaz de realizar tareas de sujeción, traslado y apilamiento de objetos. Para su construcción se utilizó una estructura fabricada mediante impresión 3D, compuesta por cinco articulaciones accionadas y una pinza, utilizando un total de seis servomotores.

El sistema electrónico se desarrolló mediante un microcontrolador ESP32, un módulo PCA9685 para el control de los servomotores y un sensor ultrasónico HC-SR04 para la detección de objetos. Además, se incorporó una fuente de alimentación externa para abastecer los actuadores.

En cuanto al software, se utilizó Python con una arquitectura Modelo–Vista–Controlador (MVC), separando los cálculos cinemáticos, la visualización del manipulador y las instrucciones de movimiento. Se implementó el método Denavit–Hartenberg para representar la geometría del brazo y calcular las transformaciones entre sus articulaciones. Asimismo, se desarrolló una interfaz de visualización tridimensional y se integró Blynk IoT para el control inalámbrico y la solicitud de rutinas automáticas.

Como resultado, el manipulador logró ejecutar la rutina de apilamiento de bloques, demostrando el funcionamiento conjunto de los componentes mecánicos, electrónicos y de programación. De esta manera, se cumplió el objetivo principal del proyecto, aplicando conocimientos de cinemática, automatización y sistemas de control robótico.

2. Introducción

Los manipuladores robóticos son utilizados en distintos procesos industriales para automatizar tareas repetitivas, como la selección, sujeción y traslado de objetos. Estos sistemas permiten realizar movimientos mediante la coordinación de sus articulaciones, reduciendo la intervención humana en determinadas operaciones [1].

En el presente proyecto se desarrolla un manipulador robótico antropomórfico a escala, construido mediante piezas impresas en 3D y accionado por seis servomotores, de los cuales cinco permiten el movimiento de las articulaciones y uno controla la apertura y cierre de la pinza. Para su funcionamiento se utiliza un microcontrolador ESP32 junto con un módulo PCA9685, encargado de generar las señales de control de los servomotores.

Además, el proyecto contempla el uso de un sensor ultrasónico HC-SR04 para la detección de objetos, una aplicación móvil mediante Blynk IoT y un programa desarrollado en Python para el control y análisis cinemático del manipulador.

El propósito del proyecto es integrar estos elementos para desarrollar un sistema capaz de detectar, sujetar, trasladar y apilar bloques de manera automática, aplicando los conocimientos adquiridos sobre cinemática, programación y sistemas de control.



Figura 1: Vista general del manipulador robótico construido.

3. Marco teórico

3.1. Manipuladores robóticos y grados de libertad

Un manipulador robótico es un mecanismo formado por eslabones y articulaciones que permiten realizar movimientos dentro de un espacio de trabajo. Estos sistemas se utilizan en tareas de manipulación, transporte y posicionamiento de objetos [1].

Los grados de libertad (GDL) corresponden a la cantidad de movimientos independientes que puede realizar un mecanismo [1]. En el caso del manipulador desarrollado, se utilizan cinco articulaciones accionadas y una pinza controlada mediante un sexto servomotor.

El alcance y la movilidad del manipulador dependen de las dimensiones de sus eslabones y de los límites mecánicos de las articulaciones. Por esta razón, es necesario considerar estos factores al momento de planificar los movimientos del brazo.

3.2. Cinemática directa e inversa

La cinemática estudia el movimiento de los cuerpos sin considerar las fuerzas que lo producen. En robótica, permite relacionar los movimientos de las articulaciones con la posición y orientación del elemento terminal.

La cinemática directa consiste en calcular la posición y orientación del extremo del manipulador a partir de los ángulos articulares conocidos.

Por otra parte, la cinemática inversa permite determinar los ángulos necesarios para alcanzar una posición determinada dentro del espacio de trabajo [1].

Ambos procedimientos son importantes para el funcionamiento del manipulador, ya que permiten

calcular su posición actual y planificar los movimientos necesarios para acercarse a un objeto, sujetarlo y trasladarlo hacia otra ubicación.

3.3. Modelo Denavit–Hartenberg

El método Denavit–Hartenberg (D–H) permite representar la geometría de un manipulador robótico mediante sistemas de coordenadas asociados a sus articulaciones.

Este método utiliza cuatro parámetros geométricos para describir la relación entre dos eslabones consecutivos [2]. En la Tabla 1 se presenta el significado de cada uno.

Parámetro	Descripción
θ_i	Ángulo de rotación de la articulación.
d_i	Desplazamiento sobre el eje z.
a_i	Distancia entre los ejes articulares.
α_i	Ángulo de torsión entre los ejes.

Tabla 1: Parámetros del método Denavit–Hartenberg.

A partir de estos parámetros se construyen matrices de transformación homogénea, que permiten relacionar la posición y orientación de las articulaciones.

La transformación entre dos sistemas de referencia consecutivos se expresa como:

$$^{i-1}T_i = R_z(\theta_i)\, T_z(d_i)\, T_x(a_i)\, R_x(\alpha_i) \tag{1}$$

Esta transformación corresponde a la convención D–H clásica [1].

Al multiplicar estas matrices de manera consecutiva, se obtiene la transformación total desde la base hasta el elemento terminal del manipulador.

4. Presentación del problema

El problema planteado consiste en desarrollar un manipulador robótico a escala que sea capaz de detectar, sujetar, trasladar y apilar objetos cúbicos conocidos de manera automática.

Para realizar estas operaciones, el manipulador debe coordinar el movimiento de sus articulaciones, permitiendo posicionar la pinza en diferentes puntos del espacio de trabajo. Esto requiere establecer una relación entre los ángulos de los servomotores y la posición del elemento terminal mediante un modelo cinemático.

Una de las principales dificultades corresponde a la precisión del posicionamiento, debido a que los movimientos deben considerar tanto la geometría del manipulador como las limitaciones mecánicas de sus articulaciones. Además, se debe conseguir una detección confiable de los objetos para ejecutar las rutinas de manipulación.

Por otra parte, el sistema debe permitir la interacción con el usuario mediante una aplicación móvil, manteniendo la coordinación entre las instrucciones recibidas, el procesamiento del programa y los movimientos físicos del brazo.

4.1. Requerimientos del sistema

Para abordar el problema planteado, se consideran los siguientes requerimientos:

- Posicionar el elemento terminal mediante el movimiento coordinado de las articulaciones.
- Relacionar los movimientos físicos del brazo con el modelo cinemático Denavit–Hartenberg.
- Establecer posiciones de referencia y límites seguros de movimiento para los servomotores.
- Detectar la presencia de objetos dentro del área de trabajo mediante un sensor.
- Permitir el control manual y la solicitud de rutinas automáticas mediante una aplicación móvil.
- Desarrollar secuencias repetibles de sujeción, traslado y apilamiento de bloques.
- Mantener una alimentación eléctrica adecuada para el funcionamiento de los actuadores y del sistema de control.

El cumplimiento de estos requerimientos permitirá evaluar la integración mecánica, electrónica y de programación del manipulador, considerando las condiciones necesarias para realizar las tareas propuestas.

5. Solución del problema

Para abordar el problema planteado, se desarrolló un manipulador robótico que integra una estructura mecánica impresa en 3D, servomotores, sensores y un sistema de control programado.

El desarrollo se dividió en etapas de construcción, integración electrónica y programación, considerando las funciones necesarias para realizar las tareas de manipulación de objetos.

5.1. Elección y construcción del manipulador

Para la estructura mecánica se seleccionó un modelo disponible en la plataforma Cults3D. Las piezas fueron fabricadas mediante impresión 3D y posteriormente ensambladas con los servomotores correspondientes.

El diseño cuenta con cinco articulaciones accionadas y una pinza, lo que permite realizar movimientos de posicionamiento y orientación del elemento terminal.



Figura 2: Vista general del manipulador robótico construido.

5.2. Elección de materiales y componentes

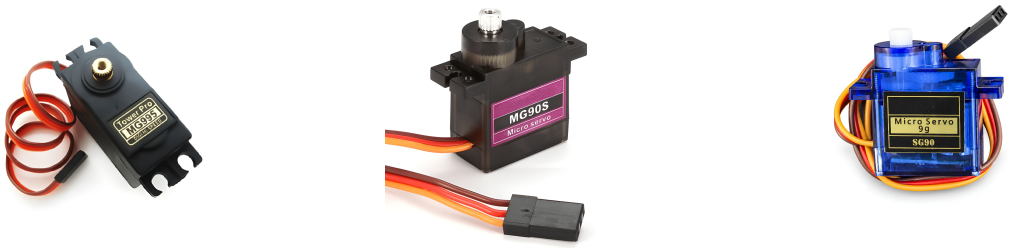
Para la construcción del manipulador se seleccionaron componentes considerando las exigencias mecánicas de las articulaciones, las necesidades de control y la detección de objetos dentro del área de trabajo.

5.2.1. Servomotores

La selección de los servomotores se realizó considerando la carga que debe soportar cada articulación del brazo. Debido a esto, se utilizaron tres modelos diferentes, distribuidos según su función.

- **MG995:** Se utilizaron tres servomotores para las articulaciones de la base, el hombro y el codo. Estas articulaciones soportan una mayor carga debido al peso de los eslabones y de los componentes ubicados hacia el extremo del brazo.
- **MG90S:** Se incorporaron dos servomotores en la muñeca del manipulador. Su menor tamaño y peso permiten controlar la orientación del elemento terminal sin incrementar excesivamente la carga sobre las articulaciones anteriores.
- **SG90:** Se utilizó un servomotor para accionar la apertura y cierre de la pinza. Debido a su tamaño reducido, permite realizar el movimiento de agarre sin agregar una masa considerable al extremo del manipulador.

En la Figura 3 se presentan los tres modelos utilizados en el proyecto.



MG995 MG90S SG90

Figura 3: Servomotores utilizados en el manipulador robótico.

5.2.2. Módulo PCA9685

El PCA9685 es un controlador de señales PWM de 16 canales y una resolución de 12 bits, que se comunica con la ESP32 mediante el protocolo I^2C [3].

Su función es generar las señales necesarias para controlar los seis servomotores de manera independiente, evitando que el microcontrolador deba generar directamente cada señal PWM.

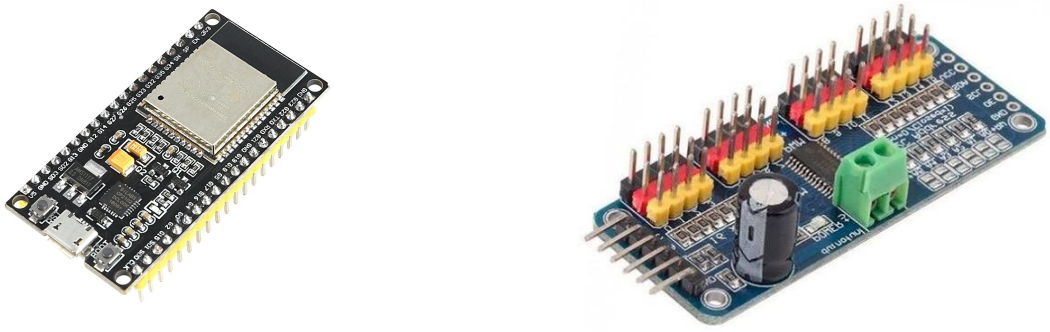
5.2.3. Microcontrolador ESP32

Se seleccionó el microcontrolador ESP32 debido a su capacidad de procesamiento y conectividad Wi-Fi integrada [4].

Dentro del sistema, la ESP32 recibe las instrucciones de movimiento y las transmite al módulo PCA9685. Además, permite integrar el sensor ultrasónico y establecer comunicación con la aplicación móvil mediante Blynk IoT.

Los cálculos cinemáticos y la organización de las rutinas se desarrollan principalmente en Python, mientras que la ESP32 se encarga de ejecutar las instrucciones sobre el hardware.

En la Figura 4 se muestran los dispositivos utilizados para el control electrónico del manipulador.



ESP32 PCA9685

Figura 4: Microcontrolador ESP32 y módulo controlador PWM PCA9685.

5.2.4. Sensor ultrasónico HC-SR04

Para detectar objetos se seleccionó el sensor ultrasónico HC-SR04, cuyo funcionamiento se basa en la emisión y recepción de ondas ultrasónicas de aproximadamente 40 kHz [5].

Se consideró inicialmente utilizar un sensor infrarrojo TCRT5000. Sin embargo, su respuesta puede variar según el color y las propiedades reflectantes de la superficie [6].

Por esta razón, se optó por el HC-SR04, ya que permite medir distancias sin depender directamente del color del objeto. No obstante, su lectura también depende de la orientación y geometría de la superficie detectada.



Figura 5: Sensor ultrasónico HC-SR04 utilizado para la detección de objetos.

5.2.5. Fuente de alimentación

Para alimentar los servomotores se dispone de una fuente externa de 5 V y 10 A.

La utilización de una fuente independiente para la etapa de potencia permite evitar que la corriente requerida por los actuadores sea suministrada directamente desde los pines de la ESP32 [7].

Para el funcionamiento del sistema se considera una referencia de tierra común entre los módulos electrónicos y la alimentación de los servomotores.

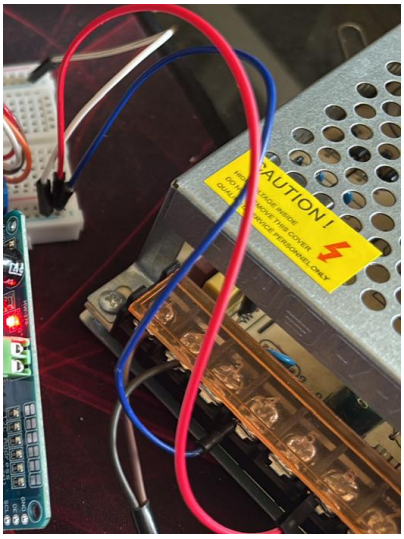


Figura 6: Fuente de alimentación externa de 5 V y 10 A utilizada en el sistema.

5.3. Integración electrónica

La integración electrónica del manipulador se realizó mediante un microcontrolador ESP32, un módulo PCA9685, seis servomotores y un sensor ultrasónico HC-SR04. Estos componentes permiten ejecutar los movimientos del brazo y obtener información del entorno.

La ESP32 se comunica con el PCA9685 mediante el protocolo I^2C. El módulo genera las señales PWM utilizadas para controlar cada servomotor de manera independiente, utilizando los canales del 10 al 15.

Para la alimentación de los servomotores se empleó una fuente externa de 5 V y 10 A. De esta manera, la corriente requerida por los actuadores no depende de los pines de alimentación del microcontrolador.

El montaje utiliza una referencia de tierra común entre los elementos que intercambian señales. Asimismo, se emplearon borneras y una placa de conexiones para distribuir la alimentación y organizar el cableado.

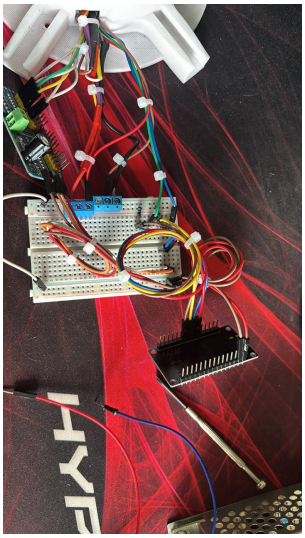


Figura 7: Montaje electrónico del manipulador robótico, con distribución de conexiones y alimentación.

En la Figura 8 se observa con mayor detalle la integración del módulo PCA9685 y la fuente externa de alimentación.

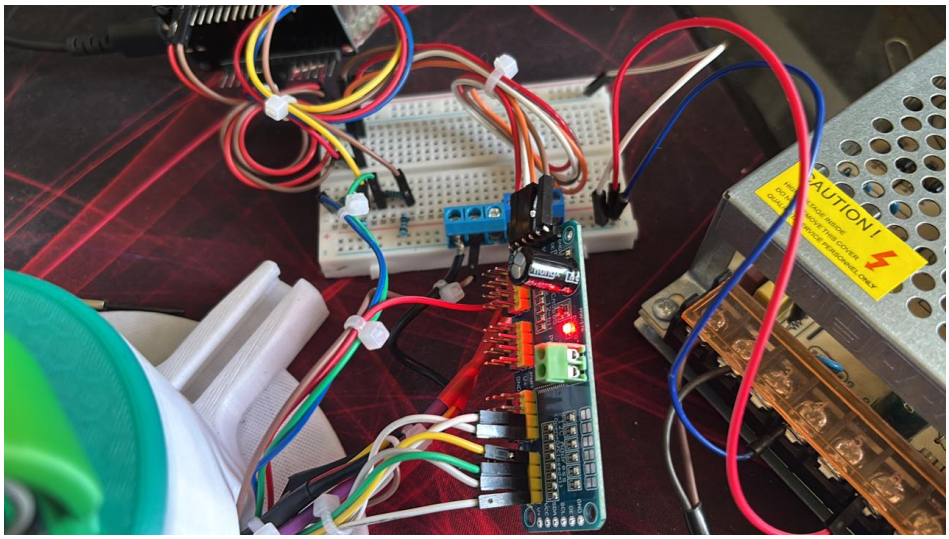


Figura 8: Detalle del módulo PCA9685 y de las conexiones con la fuente de alimentación.

Por otra parte, el sensor ultrasónico HC-SR04 se conecta a la ESP32 mediante los pines TRIG y ECHO. Debido a que la señal ECHO puede alcanzar 5 V, se incorporó un divisor resistivo para adaptar su tensión al nivel lógico de 3,3 V del microcontrolador [5, 4].

Esta integración permite que la ESP32 reciba instrucciones desde Python o Blynk IoT, controle los movimientos de los servomotores y gestione las lecturas del sensor.

5.4. Arquitectura del sistema de control

El software del manipulador fue desarrollado principalmente en Python y organizado mediante el patrón Modelo–Vista–Controlador (MVC). Esta estructura permite separar los cálculos cinemáticos, la interfaz gráfica y las instrucciones de movimiento [8].

El modelo contiene los archivos `dh.py`, `inversa.py`, `geometria.py` y `config.py`. Estos módulos reúnen los parámetros geométricos, los cálculos de posición y las operaciones necesarias para representar el manipulador.

En particular, el módulo `dh.py` implementa el método Denavit–Hartenberg, mediante el cual se obtienen las matrices de transformación correspondientes a las cinco articulaciones del brazo.

A partir de estas matrices se calcula la transformación total desde la base hasta el elemento terminal, permitiendo conocer su posición y orientación según los ángulos articulares.

La cinemática inversa, implementada en `inversa.py`, permite calcular configuraciones articulares para alcanzar posiciones determinadas, considerando las restricciones definidas en el modelo.

La vista está formada principalmente por los archivos `gui.py` y `dibujo.py`. Su función es representar gráficamente el manipulador, visualizar las articulaciones y mostrar los parámetros y matrices obtenidos mediante el modelo D–H.

Por otra parte, el controlador coordina las solicitudes de movimiento, la comunicación serial con la ESP32 y la ejecución de las rutinas automáticas.

La estructura del programa se representa mediante las clases principales:

- **Interfaz:** Administra la ventana gráfica de Python y presenta la información del sistema.
- **Escena3D:** Actualiza la representación tridimensional del manipulador.
- **Controlador:** Coordina los cálculos cinemáticos, la comunicación y las instrucciones de movimiento.
- **Rutina:** Organiza las secuencias automáticas de manipulación y desplazamiento.
- **ESP32:** Clase de Python encargada de gestionar la comunicación serial con el microcontrolador físico.

En la Figura 9 se presenta el diagrama UML correspondiente a las clases principales del programa.

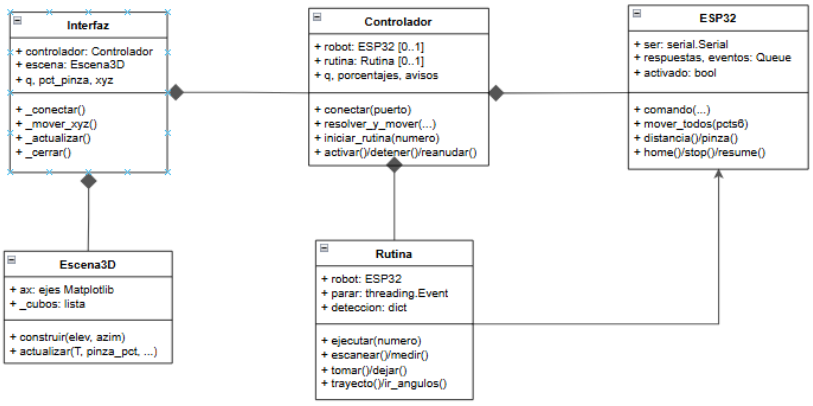


Figura 9: Diagrama UML de clases de la aplicación desarrollada para el manipulador robótico.

La separación de responsabilidades facilita la organización del código y permite modificar los cálculos, la visualización o las rutinas sin concentrar toda la programación en un único archivo.

5.5. Interfaz de visualización y rutinas automáticas

Para visualizar el funcionamiento del manipulador se desarrolló una interfaz gráfica en Python, que permite observar el brazo en tres dimensiones y consultar la información de su modelo cinemático.

La representación muestra las cinco articulaciones, sus ejes de referencia y la posición calculada del elemento terminal. También permite visualizar los parámetros Denavit–Hartenberg y las matrices de transformación correspondientes a cada articulación.

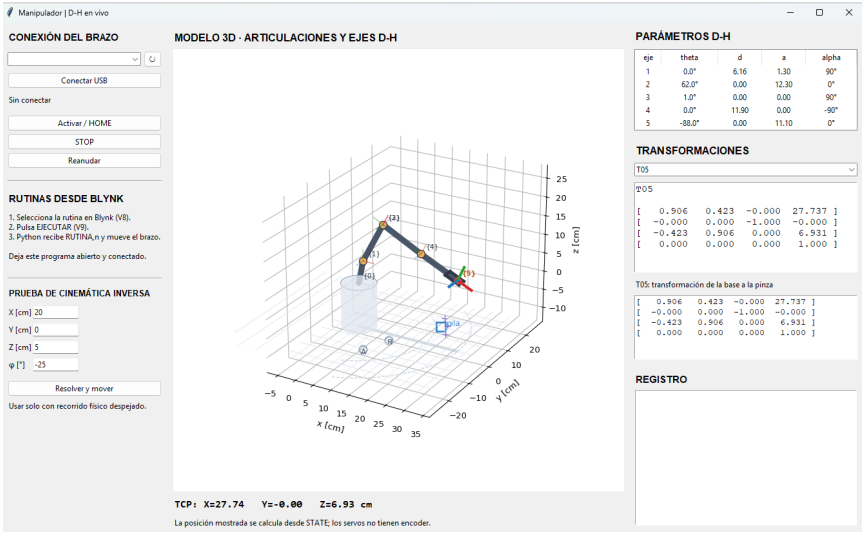


Figura 10: Interfaz de visualización en Python, con modelo tridimensional, parámetros D–H y matrices de transformación.

La posición mostrada en la interfaz se obtiene mediante los cálculos cinemáticos realizados en Python. Por lo tanto, corresponde a una posición calculada a partir de los valores articulares, y no a una medición directa mediante encoders externos.

5.5.1. Aplicación Blynk IoT

Para el control inalámbrico se utilizó Blynk IoT, permitiendo enviar instrucciones a la ESP32 mediante una conexión Wi-Fi [9].

La aplicación dispone de controles individuales para las cinco articulaciones y la pinza. Además, permite configurar la velocidad, seleccionar una rutina y solicitar su ejecución.

También se incorporaron indicadores para visualizar las coordenadas del elemento terminal, la distancia registrada por el sensor ultrasónico y el estado de la rutina.

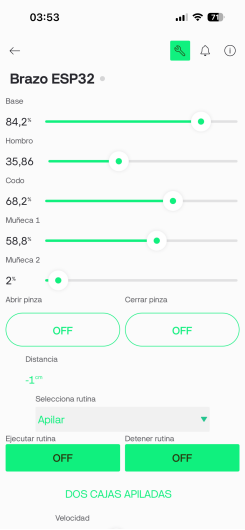


Figura 11: Interfaz móvil Blynk IoT utilizada para interactuar con el manipulador.

Los mismos controles e indicadores se organizaron en un panel web, facilitando la interacción con el sistema desde un computador.

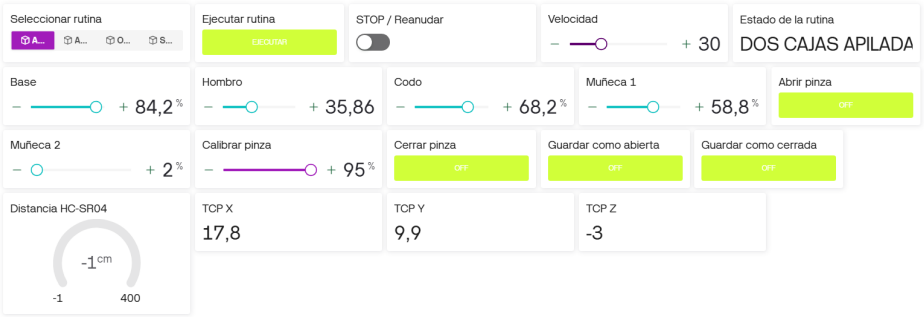


Figura 12: Panel web de Blynk IoT con controles articulares y selección de rutinas.

5.5.2. Ejecución de rutinas automáticas

Las rutinas se programaron en Python y se organizaron dentro del archivo `rutinas.py`. Estas incluyen movimientos articulares, detección de objetos mediante el sensor ultrasónico, accionamiento de la pinza y traslado de bloques.

Cuando se solicita una rutina desde Blynk, la ESP32 comunica la instrucción al programa Python mediante la conexión serial USB. Posteriormente, Python ejecuta la secuencia programada y envía las órdenes necesarias al microcontrolador.

La ESP32 recibe estas instrucciones y utiliza el PCA9685 para generar las señales PWM que controlan los servomotores del brazo.

5.5.3. Resultado de la rutina de apilamiento

Durante las pruebas físicas se consiguió ejecutar la rutina de apilamiento de dos bloques. El manipulador realizó los movimientos de aproximación, sujeción, elevación y traslado, hasta colocar los objetos en sus respectivas posiciones.

Para esta tarea se utilizaron posiciones articulares previamente definidas, almacenadas en el archivo `apilado_ensenado.json`. Esto permitió establecer los movimientos necesarios entre los puntos de recogida y depósito.

Durante los ensayos fue necesario ajustar la posición de liberación del segundo bloque. Se incorporó un desplazamiento de 5 mm alejándose de la base del manipulador, lo que permitió mejorar la alineación de ambas cajas durante el apilamiento.

En la Figura 13 se muestra el brazo durante la ejecución de esta tarea.

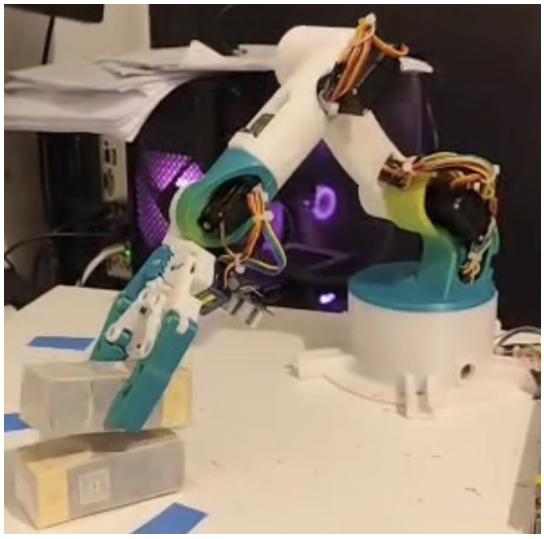


Figura 13: Manipulador robótico durante la rutina de manipulación y apilamiento de bloques.

Finalmente, el manipulador logró completar el apilamiento de los dos bloques. Este resultado permitió comprobar el funcionamiento de la secuencia programada y la coordinación entre los servomotores, el

sensor y el sistema de control.

5.6. Organización del equipo

Para llevar a cabo el desarrollo del manipulador robótico, se establecieron cuatro áreas de trabajo, con el propósito de distribuir las responsabilidades según las distintas necesidades del proyecto.

Las áreas consideradas corresponden a integración técnica, investigación y modelado, software y automatización, y gestión de mercado. Cada una cuenta con responsables definidos, permitiendo organizar las actividades de construcción, programación, investigación y selección de componentes.

En la Tabla 2 se presenta la distribución de responsabilidades del equipo.

Área	Responsabilidades	Integrante(s)
Técnica e integración	Montaje mecánico, conexiones eléctricas e integración de los componentes.	Alejandro Pavez
Investigación y modelado	Investigación teórica, modelado cinemático y método D–H.	Luciana Bravo
Software y automatización	Programación en Python y ESP32, aplicación Blynk y rutinas automáticas.	Alejandro Pavez, Luciana Bravo y Diego Zúñiga
Gestión y mercado	Evaluación de componentes, proveedores y costos del proyecto.	Diego Zúñiga

Tabla 2: Distribución de responsabilidades del equipo de trabajo.

La distribución de áreas permitió establecer responsabilidades individuales y compartidas. En particular, el desarrollo del software y la automatización se planteó como una actividad conjunta entre los tres integrantes, debido a su relación con las demás etapas del proyecto.

6. Conclusiones

El proyecto permitió desarrollar un manipulador robótico funcional, capaz de sujetar, trasladar y apilar dos bloques de manera automática. Para alcanzar este resultado fue necesario integrar la estructura mecánica, los componentes electrónicos y las rutinas programadas, realizando ajustes durante las pruebas físicas.

La estructura fabricada mediante impresión 3D incorporó cinco articulaciones accionadas y una pinza, utilizando seis servomotores. La selección de los actuadores permitió distribuirlos según las necesidades de cada parte del brazo, considerando las cargas y movimientos requeridos.

En el sistema electrónico se utilizó una ESP32 conectada al módulo PCA9685, encargado de generar las señales de control para los servomotores. También se incorporó el sensor ultrasónico HC-SR04 para apoyar la detección de los objetos durante las rutinas automáticas.

Respecto al software, la organización mediante el patrón Modelo–Vista–Controlador permitió separar los cálculos cinemáticos, la representación tridimensional y las funciones de control. Se implementaron la cinemática directa mediante Denavit–Hartenberg y la cinemática inversa, utilizando estos cálculos dentro del programa desarrollado en Python.

Además, se completó la integración con Blynk IoT, permitiendo controlar las articulaciones y solicitar rutinas mediante una aplicación móvil. Esto proporcionó una alternativa de interacción inalámbrica con el manipulador.

Uno de los aspectos más importantes del desarrollo fue la calibración de las posiciones utilizadas en el apilamiento. Durante las pruebas se ajustó la ubicación de depósito del segundo bloque para conseguir una mejor alineación, demostrando la importancia de adaptar los movimientos programados al comportamiento físico del brazo.

Finalmente, se cumplió el objetivo principal del proyecto al completar la rutina de apilamiento de dos bloques. El trabajo permitió aplicar los contenidos de cinemática, programación y control estudiados en la asignatura de Fundamentos de Robótica.

Referencias

[1] Craig, J. J. (2018). *Introduction to Robotics: Mechanics and Control* (4.a ed.). Pearson.

[2] Denavit, J. y Hartenberg, R. S. (1955). A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices. *Journal of Applied Mechanics*, 22(2), 215–221. DOI: 10.1115/1.4011045.

[3] NXP Semiconductors. (2015). *PCA9685: 16-channel, 12-bit PWM Fm+ I2C-bus LED controller* (Rev. 4). Hoja de datos. https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf

[4] Espressif Systems. (s. f.). *ESP32: Wi-Fi & Bluetooth SoC*. Documentación oficial del fabricante. https://www.espressif.com/en/products/socs/esp32

[5] ElecFreaks. (s. f.). *Ultrasonic Ranging Module HC-SR04*. Hoja de datos. https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[6] Vishay Semiconductors. (s. f.). *TCRT5000, TCRT5000L: Reflective Optical Sensor with Transistor Output*. Documentación técnica. https://www.vishay.com/en/product/83760/

[7] Earl, B. (2012). *Adafruit PCA9685 16-Channel Servo Driver: Hooking It Up*. Adafruit Learning System. https://learn.adafruit.com/16-channel-pwm-servo-driver/hooking-it-up

[8] Fowler, M. (2003). *Model View Controller*. Patterns of Enterprise Application Architecture. https://martinfowler.com/eaaCatalog/modelViewController.html

[9] Blynk. (s. f.). *Control Devices (GPIOs and Beyond)*. Documentación oficial de Blynk IoT. https://docs.blynk.io/en/getting-started/using-virtual-pins-to-control-physical-devices

Anexo I: Rutina de código

En este anexo se presenta la rutina de apilamiento de dos bloques, implementada en Python mediante el archivo `rutinas.py`. Esta secuencia coordina los movimientos articulares, la detección de objetos y el accionamiento de la pinza para realizar la tarea de apilamiento.

```
    def _rutina_apilado_ensenado(self):
        # Reproduce la trayectoria A-B validada físicamente para exactamente dos cajas.
        datos = self._cargar_apilado_ensenado()
        m = datos["traslado_a_b"]
        # Las calibraciones estan en el JSON, no duplicadas en el ESP32.
        self.robot.configurar_pinza(float(datos["pinza_abierta"]),
                                    float(datos["pinza_cerrada"]))
        self.robot.velocidad(int(datos.get("velocidad", 30)))

        # Primero se alcanza la postura alta más cercana. Esto evita barrer la mesa
        # si la rutina comienza cerca de A o cerca de la pila B.
        qa = dh.porcentajes_a_angulos(m["a_alta_porcentajes"])
        qb = dh.porcentajes_a_angulos(m["b_alta_porcentajes"])
        distancia_a = sum(abs(a - b) for a, b in zip(self.q, qa))
        distancia_b = sum(abs(a - b) for a, b in zip(self.q, qb))
        empieza_en_b = distancia_b < distancia_a
        q_alta = qb if empieza_en_b else qa
        self.ir_articular(q_alta, pasos=8)
        if empieza_en_b:
            self.ir_articular(qa, pasos=10)

        for numero in (1, 2):
            self.avisar("progreso", (numero - 1) / 2)
            self._ir_ensenado(m, "a_aproximacion_sensor", pasos=6)
            distancia = self._esperar_caja_en_a(datos, m, numero)
            self._log(f"  Caja {numero} confirmada a {distancia:.2f} cm")

            self.pinza(True)
            self._ir_ensenado(m, "a_aproximacion_agarre", pasos=3)
            self._ir_ensenado(m, "a_agarre", pasos=6)
            self.pinza(False)
            self._reportar(f"TRASLADANDO CAJA {numero}")
            self._ir_ensenado(m, "a_elevacion_1", pasos=3)
            self._ir_ensenado(m, "a_elevacion_2", pasos=3)
            self._ir_ensenado(m, "a_elevacion_3", pasos=3)
            self._ir_ensenado(m, "a_alta", pasos=4)
            self._ir_ensenado(m, "b_alta", pasos=10)

            if numero == 1:
                self._ir_ensenado(m, "b_aproximacion", pasos=5)
                self._ir_ensenado(m, "b_deposito", pasos=6)
                self.pinza(True)
                self._ir_ensenado(m, "b_aproximacion", pasos=5)
                self._ir_ensenado(m, "b_alta", pasos=5)
                self._ir_ensenado(m, "a_alta", pasos=10)
            else:
                self._ir_ensenado(m, "b_segundo_nivel_aproximacion", pasos=5)
                self._ir_ensenado(m, "b_segundo_nivel_precolocacion", pasos=5)
                # Este último punto incorpora los 5 mm alejándose de la base que
                # dejaron ambas cajas alineadas en el ensayo físico.
                self._ir_ensenado(m, "b_segundo_nivel_suelta", pasos=3)
                self.pinza(True)
                self._ir_ensenado(m, "b_segundo_nivel_aproximacion", pasos=5)
                self._ir_ensenado(m, "b_alta", pasos=5)

        self.avisar("progreso", 1.0)
        self._reportar("DOS CAJAS APILADAS")
```

Listing 1: Rutina de apilamiento de dos bloques en Python.

La rutina utiliza posiciones previamente definidas en el archivo `apilado_ensenado.json`, junto con funciones auxiliares del programa para ejecutar los movimientos.

Anexo II: Desarrollo del algoritmo Denavit–Hartenberg

En este anexo se presenta el desarrollo del modelo Denavit–Hartenberg implementado en Python para el manipulador robótico. El modelo considera cinco articulaciones rotacionales, mientras que el servomotor de la pinza se controla de manera independiente.

Parámetros geométricos

Para representar el manipulador se definieron las dimensiones de sus eslabones, expresadas en centímetros. Estos valores corresponden a los parámetros utilizados en el archivo `dh.py`.

Parámetro	Descripción	Valor (cm)
H	Altura del hombro	6,16
R_0	Separación horizontal de la base	1,30
L_1	Longitud hombro–codo	12,30
L_2	Longitud codo–muñeca	11,90
L_G	Longitud muñeca–pinza	11,10

Tabla 3: Dimensiones utilizadas en el modelo cinemático.

La altura del hombro se obtuvo mediante la siguiente relación geométrica:

$$H = \sqrt{6,3^2 - 1,3^2} \tag{2}$$

$$H \approx 6,16 \text{ cm} \tag{3}$$

Tabla de parámetros D–H

A partir de la geometría del manipulador se establecieron los parámetros Denavit–Hartenberg correspondientes a cada una de las cinco articulaciones.

Eje	θ_i	d_i	a_i	α_i
1	q_1	H	R_0	90°
2	q_2	0	L_1	0°
3	$q_3 + 90^\circ$	0	0	90°
4	q_4	L_2	0	-90°
5	$q_5 - 90^\circ$	0	L_G	0°

Tabla 4: Parámetros D–H utilizados en el manipulador robótico.

Los valores q_1 a q_5 corresponden a los ángulos articulares del manipulador. Los desplazamientos angulares de las articulaciones 3 y 5 se incorporan para representar la orientación de sus sistemas de

referencia.

Matriz de transformación

Para cada articulación se construye una matriz de transformación homogénea mediante la convención D–H estándar [1]:

$$ {}^{i-1}T_i = \begin{bmatrix} c_\theta & -s_\theta c_\alpha & s_\theta s_\alpha & a_i c_\theta \\ s_\theta & c_\theta c_\alpha & -c_\theta s_\alpha & a_i s_\theta \\ 0 & s_\alpha & c_\alpha & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4} $$

Donde $c_\theta = \cos(\theta_i)$, $s_\theta = \sin(\theta_i)$, $c_\alpha = \cos(\alpha_i)$ y $s_\alpha = \sin(\alpha_i)$.

La transformación total se calcula mediante la multiplicación consecutiva de las cinco matrices:

$$ {}^0T_5 = {}^0T_1 \, {}^1T_2 \, {}^2T_3 \, {}^3T_4 \, {}^4T_5 \tag{5} $$

La matriz resultante contiene la orientación y posición del elemento terminal respecto a la base del brazo.

Las coordenadas cartesianas se obtienen desde la última columna de la matriz:

$$ \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} T_{14} \\ T_{24} \\ T_{34} \end{bmatrix} \tag{6} $$

Ejemplo de cálculo

Para comprobar el funcionamiento del modelo se considera la configuración angular de referencia utilizada durante la calibración:

$$ \mathbf{q} = \begin{bmatrix} 0^\circ & 62^\circ & -89^\circ & 0^\circ & 2^\circ \end{bmatrix}^T \tag{7} $$

Al sustituir estos valores en el modelo y multiplicar las matrices, se obtiene aproximadamente:

$$ {}^0T_5 \approx \begin{bmatrix} 0,9063 & 0,4226 & 0 & 27,74 \\ 0 & 0 & -1 & 0 \\ -0,4226 & 0,9063 & 0 & 6,93 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{8} $$

Por lo tanto, la posición calculada del centro de la pinza corresponde a:

$$\boxed{(x, y, z) \approx (27,74;\ 0;\ 6,93)\ \text{cm}} \tag{9}$$

Este resultado representa la posición obtenida matemáticamente para la configuración angular indicada.

Implementación en Python

El modelo Denavit–Hartenberg se implementó en el archivo `dh.py`, donde se definen los parámetros geométricos y las funciones necesarias para calcular las matrices de transformación de las cinco articulaciones.

La función `matrices()` obtiene las transformaciones individuales y acumuladas, mientras que `matriz_total()` calcula la transformación desde la base hasta el elemento terminal. A partir de esta matriz, la función `cinematica_directa()` determina las coordenadas cartesianas del centro de la pinza.

Estos resultados se integran en la interfaz gráfica mediante los archivos `gui.py` y `dibujo.py`, permitiendo visualizar los parámetros D–H, las matrices de transformación y la representación tridimensional del manipulador.

Anexo III: Repositorio GitHub

El código fuente completo del manipulador robótico se encuentra disponible en un repositorio de GitHub, donde se incluyen los archivos desarrollados durante el proyecto.

El repositorio contiene el programa de control en Python, organizado mediante la arquitectura Modelo–Vista–Controlador (MVC), los módulos de cinemática directa e inversa, el algoritmo Denavit–Hartenberg y las rutinas automáticas.

Además, incorpora el firmware de la ESP32, la integración con Blynk IoT y los archivos de configuración necesarios para el sistema.

El proyecto puede consultarse mediante el siguiente enlace:

https://github.com/AlePA8/Manipulador